



OpenEoX Core Schema Version 1.0

Committee Specification Draft 01

13 July 2026

This Stage

<https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/eox-core-v1.0-csd01.md> (Authoritative)

<https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/eox-core-v1.0-csd01.html>

<https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/eox-core-v1.0-csd01.pdf>

Previous Stage

N/A

Latest Stage

<https://docs.oasis-open.org/openeox/eox-core/v1.0/eox-core-v1.0.md> (Authoritative)

<https://docs.oasis-open.org/openeox/eox-core/v1.0/eox-core-v1.0.html>

<https://docs.oasis-open.org/openeox/eox-core/v1.0/eox-core-v1.0.pdf>

Technical Committee

OASIS OpenEoX TC

Chairs

Justin Murphy (justin.murphy@cisa.dhs.gov), [DHS Cybersecurity and Infrastructure Security Agency \(CISA\)](#)

Omar Santos (osantos@cisco.com), [Cisco Systems](#)

Secretaries

Stefan Hagen (stefan@hagen.link), [Individual](#)

Editors

Jautau White (jaywhite@microsoft.com), [Microsoft Corporation](#)

Stefan Hagen (stefan@hagen.link), [Individual](#)

Thomas Schmidt (thomas.schmidt@bsi.bund.de), [Federal Office for Information Security \(BSI\) Germany](#)

Additional Artifacts

This prose specification is one component of a Work Product that also includes:

- OpenEoX Core JSON schema: <https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/schema/core.json>.
Latest stage: <https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/core.json>.
- OpenEoX Meta JSON schema: <https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/schema/meta.json>.
Latest stage: <https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/meta.json>.

Declared JSON namespaces

- <https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/core.json>
- <https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/meta.json>

Abstract

The OpenEoX Core Schema defines the core schema for the OpenEoX unified machine-readable approach to managing and sharing General Availability (GA), End-of-Sales (EoS), End-of-Life (EoL), and End-of-Security-Support (EoSSec) information for commercial and open source software and hardware.

Citation Format

When referencing this document, the following citation format should be used:

[EoX-Core-v1.0]

OpenEoX Core Schema Version 1.0. Edited by Jautau White, Stefan Hagen, and Thomas Schmidt. 13 July 2026. OASIS Committee Specification Draft 01. <https://docs.oasis-open.org/openeox/eox-core/v1.0/csd01/eox-core-v1.0-csd01.html>. Latest stage: <https://docs.oasis-open.org/openeox/eox-core/v1.0/eox-core-v1.0.html>

Related Work

This document replaces or supersedes:

N/A

This document is related to:

N/A

License, Document Status, and Notices

Copyright © OASIS Open 2026. All Rights Reserved.

For license and copyright information, and complete status, please see [Annex A](#) which contains the License, Document Status and Notices.

Table of Contents

1. Scope	4
2. Definitions and Acronyms	5
2.1 Abbreviations and Acronyms	5
2.2 Definitions	5
3. Document Conventions	8
3.1 Key Words	8
3.2 Typographical Conventions	8
4. Introduction	9
4.1 Design Considerations	9
4.2 Changes From the Previous Version	12
5. Taxonomy	13
5.1 End-of-Life	13
5.2 End-of-Sales	13
5.3 End-of-Security-Support	13
5.4 General Availability	13
5.5 Product	13
5.6 Product Life Cycle	13
5.7 Vendor	13
6. Schema Elements	14
6.1 Definitions	14
6.2 Properties	14
7. Tests	16
7.1 Mandatory Tests	16
7.2 Recommended Tests	17
7.3 Informative Tests	18
8. Safety, Security and Data Protection	19
9. Conformance	20
9.1 Conformance Targets	20
Annex A. License, Document Status and Notices	25
A.1 Document Status	25
A.2 License and Notices	25
Annex B. References	27
B.1 Normative References	27
B.2 Informative References	27
Appendix 1. Acknowledgments	29
Leadership	29
Special Thanks	29
Participants	29
Appendix 2. Changes From Previous Version	30
Revision History	30

1. Scope

This specification defines the OpenEoX Core Schema - a minimal, machine-readable JSON format for representing product life cycle milestone information for commercial and open source software and hardware products. The schema provides a common vocabulary for four key life cycle events: General Availability (GA), End-of-Sales (EoS), End-of-Life (EoL), and End-of-Security-Support (EoSSec).

The OpenEoX Core Schema is designed to be imported by or integrated into other standards, specifications, and product-specific implementations, enabling consistent and interoperable exchange of life cycle information across vendor ecosystems and consumer applications.

Product identification - the binding of life cycle milestone information to a specific product - is addressed by complementary specifications such as OpenEoX Shell and is therefore out of scope of this document.

2. Definitions and Acronyms

2.1 Abbreviations and Acronyms

This document uses the following abbreviations and acronyms:

- EoL: End-of-Life.
- EoS: End-of-Sales.
- EoSSec: End-of-Security-Support.
- EoX: End-of-X.
- GA: General Availability.

2.2 Definitions

2.2.1 Terms Defined Elsewhere

This document uses the following terms defined elsewhere:

Embedded Link [CSAF-v2.1]

syntactic construct which enables a message string to refer to a location mentioned in the document.

Empty Array: [CSAF-v2.1]

array that contains no elements, and so has a length of zero.

Empty Object [CSAF-v2.1]

object that contains no properties.

Empty String [CSAF-v2.1]

string that contains no characters, and so has a length of zero.

(End) User [CSAF-v2.1]

person who uses the information in a document to investigate, triage, or resolve results.

Product [CSAF-v2.1]

is any deliverable (e.g. software, hardware, specification, or service) which can be referred to with a name. This applies regardless of the origin, the license model, or the mode of distribution of the deliverable.

Property [CSAF-v2.1]

attribute of an object consisting of a name and a value associated with the name.

Triage [CSAF-v2.1]

decide whether a result indicates a problem that needs to be corrected.

User [CSAF-v2.1]

see end user.

Vendor [CSAF-v2.1]

the community, individual, or organization that created or maintains a product (including open source software and hardware providers).

White Space [CSAF-v2.1]

code point used to improve text readability or token separation as defined in section 12.2 of [ECMA-262].

2.2.2 Terms Defined in this Document

This document defines the following terms:

Comparator

A program that compares EoX Core Information and returns the newer one.

Consumer

A program that reads and interprets EoX Core Information.

End-of-Life (EoL)

indicates the last day when the particular product (or the product version/release) is officially supported in any way by the vendor.

End-of-Sales (EoS)

indicates the last day when a particular product (or the product version/release) may be ordered by customers from vendor sales channels.

End-of-Security-Support (EoSsec)

indicates the last day when the vendor has committed to providing security remediations for the particular product (or the product version/release).

EoX Core Information

Information about end-of-everything in the format defined by this document.

Differ

A program that compares EoX Core Information and calculates the changes.

General Availability (GA)

indicates the first day when the particular product (or the product version/release) is officially launched and made accessible to the general public or through its intended distribution channels.

Library

A library that implements OpenEoX Core data capabilities.

Library with Basic Validation

A Library that also satisfies the conformance target “Validator Basic”.

Library with Extended Validation

A Library that also satisfies the conformance target “Validator Extended”.

Library with Full Validation

A Library that also satisfies the conformance target “Validator Full”.

Merger

A program that combines EoX Core Information.

Producer

A program which emits output in the OpenEoX Core format.

Product Life Cycle

describes for a product type (software, hardware, managed service and other deliverables) the model it can be associated with. It can contain definitions of various support models (different levels of maintenance) in association to the product versioning convention.

Redactable Property

property that potentially contains sensitive information that an EoX Producer might wish to redact.

Sorter

A program that sorts EoX Core Information from newest to oldest or vice versa.

Taxonomy

classification of product life cycle stages into a set of categories.

Validator Basic

A program that reads a JSON object and checks it against the JSON schema and performs mandatory tests.

Validator Extended

A Validator Basic that additionally performs recommended tests.

Validator Full

A Validator Extended that additionally performs guidance tests.

Viewer

A Consumer that reads EoX Core Information, displays a list of the results it contains, and allows an end user to view each result in the context of the artifact in which it occurs.

Viewer Recommending Actions

A Viewer that provides actionable interpretations by mapping life cycle events to recommended actions.

3. Document Conventions

3.1 Key Words

The key words “**MUST**”, “**MUST NOT**”, “**REQUIRED**”, “**SHALL**”, “**SHALL NOT**”, “**SHOULD**”, “**SHOULD NOT**”, “**RECOMMENDED**”, “**NOT RECOMMENDED**”, “**MAY**”, and “**OPTIONAL**” in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

3.2 Typographical Conventions

Keywords defined by this specification use this `monospaced font`.

Normative source code uses this paragraph style.

Some sections of this specification are illustrated with non-normative examples introduced with “Example” or “Examples” like so:

Example 1:

Informative examples also use this paragraph style but preceded by the text "Example(s)".

All examples in this document are informative only.

All other text is normative unless otherwise labeled e.g. like the following informative comment:

This is a pure informative comment that may be present, because the information conveyed is deemed useful advice or common pitfalls learned from implementer or operator experience and often given including the rationale.

4. Introduction

4.1 Design Considerations

The OpenEoX provides the vocabulary for exchanging standardized End-of-X (EoX) life cycle data for any product. This includes, but is not limited to hardware, software and specifications, as well as all supplier-consumer relationships, services and offerings.

4.1.1 OpenEoX Architecture

The OpenEoX standard is architected as a modular three-layer framework:

4.1.1.1 OpenEoX Core (this specification)

The OpenEoX Core specification contains the fundamental EoX life cycle information elements and is designed to be imported by other standards or integrated directly into product responses. The Core schema establishes one part of the essential data elements required for any EoX statement:

Required fields:

- `end_of_life`: The definitive timestamp when all vendor support ceases
- `end_of_security_support`: The critical timestamp when security remediation commitments end
- `last_updated`: The timestamp tracking for data freshness and change management

Optionally, the standard also conveys additional life cycle events.

The core information can be provided independently when the product context is available through other means, such as SNMP responses, HTTPS API responses.

4.1.1.2 OpenEoX Shell

The OpenEoX Shell specification provides the binding mechanism that combines OpenEoX Core life cycle information to specific product identifications. This combination is called an “OpenEoX statement” - a complete, standalone representation of life cycle information for a specific product. The Shell forms the standalone mode in OpenEoX, enabling complete life cycle statements that can be processed independently without external context.

4.1.1.3 OpenEoX API

The OpenEoX API specification addresses the distribution, discovery, and consumption of OpenEoX Core and Shell data across vendor ecosystems and consumer applications. The API specification defines standardized endpoints, query mechanisms, and data exchange patterns that enable automated integration.

4.1.2 Design Principles

4.1.2.1 Standardization and Interoperability

OpenEoX addresses the current industry challenge where vendors use proprietary, inconsistent formats for communicating life cycle information. By establishing a common JSON-based format, OpenEoX enables:

- Automated processing and integration across vendor ecosystems
- Consistent interpretation of life cycle stages regardless of vendor
- Reduced operational overhead in managing multi-vendor environments
- Enhanced accuracy in infrastructure life cycle planning

4.1.2.2 Security-First Approach

Recognition that End-of-Security-Support represents a critical inflection point for infrastructure security has influenced the OpenEoX design. The mandatory `end_of_security_support` field ensures that security life cycle information is always available, supporting:

- Proactive vulnerability management planning
- Compliance with security frameworks requiring current support status
- Risk assessment for critical infrastructure components
- Automated alerting when security support approaches termination

4.1.3 Industry Applications

4.1.3.1 Vendor Perspectives

OpenEoX enables vendors to:

- Standardize EoX communication across all product lines and channels
- Reduce support inquiries related to life cycle confusion
- Demonstrate proactive customer stewardship through clear life cycle communication
- Support automated customer notification systems
- Facilitate integration with partner and reseller systems

4.1.3.2 Consumer Perspectives

Organizations consuming OpenEoX information benefit through:

- Automated discovery of life cycle status across multi-vendor environments
- Proactive planning for product refreshes and migrations
- Enhanced compliance reporting for security and regulatory frameworks
- Reduced operational risk from unexpected support terminations
- Streamlined asset management and inventory tracking

The OpenEoX architecture ensures that organizations can adopt OpenEoX in a manner that aligns with their existing infrastructure and operational models while preserving the ability to enhance integration over time.

4.1.4 Construction Principles

The OpenEoX Core specification describes a map for well-known named transition events (like end-of-life) to dates. Producers use this mapping to bind products to this minimal set of life cycle events.

Other schemata provide models to add the data needed for product identification. Examples for such schemata are:

- OpenEoX Shell
- OpenEoX API
- integration into the product itself (for example via SNMP calls or HTTPS requests)
- integration into other standards (for example CSAF)

This separation provides consistent, interoperable, actionable, structured, and validated life cycle information with the goal to offer complete automation.

The chosen data format and version “JSON Schema Draft 2020-12” [[JSON-Schema-Core](#)] allows validation and delegation to providers of referenced schemata. The latter enables separation of concerns as it allows other standards, specifications, and product-specific implementations to import and reference the OpenEoX Core schema. This enables consistency and interoperability across different implementations.

The OpenEoX Core schema is defined at <https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/core.json>

The schema elements and their expected usage patterns are detailed in subsequent sections.

Linking and integration of the OpenEoX Core schema with product identification schema information may lead to inconsistencies. The producers of OpenEoX information items have to ensure the intended semantics. For example, that life cycle dates are ordered logically. Semantic validation is out of scope for this OpenEoX Core specification.

The OpenEoX Core schema is designed to be self-contained with minimal external dependencies, utilizing only:

[[RFC3339](#)]

Date and time format specification for timestamp representation

[[JSON-Schema-Core](#)]

Schema validation and structure definition

OpenEoX information items SHOULD only contain members where the value is an OpenEoX life cycle date. The use of placeholders or `null` values is discouraged and any compliant OpenEoX implementation SHOULD ignore such members.

In combination, these two preceding requirements maximize clarity and simplify automation.

OpenEoX information items SHOULD NOT contain additional properties in themselves or as part of referenced schema instances. Suggestions for new fields or values SHOULD be made through issues in the TC's GitHub. The JSON schemas defined in this standard do not allow the use of additional properties and custom keywords.

The standardized fields allow for scalability across different issuing parties and dramatically reduce the human effort and need for dedicated parsers as well as other tools on the side of the consuming parties.

4.1.5 Format Validation

The JSON schema 2020-12 dialect per default uses the `format` keyword just as annotation. To be able to ensure that the format constraints are validated as intended, the following metaschema is defined.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/meta.json",
  "$dynamicAnchor": "meta",
  "$vocabulary": {
    "https://json-schema.org/draft/2020-12/vocab/core": true,
    "https://json-schema.org/draft/2020-12/vocab/applicator": true,
    "https://json-schema.org/draft/2020-12/vocab/unevaluated": true,
    "https://json-schema.org/draft/2020-12/vocab/validation": true,
    "https://json-schema.org/draft/2020-12/vocab/meta-data": true,
    "https://json-schema.org/draft/2020-12/vocab/format-assertion": true,
    "https://json-schema.org/draft/2020-12/vocab/content": true
  },
  "allOf": [
    { "$ref": "https://json-schema.org/draft/2020-12/meta/core" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/applicator" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/unevaluated" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/validation" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/meta-data" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/format-assertion" },
    { "$ref": "https://json-schema.org/draft/2020-12/meta/content" }
  ]
}
```

All vocabularies that can be used are defined in this metaschema as JSON schema 2020-12 dialect has the rule of non-inheritability of vocabularies.

It is then consequently used in all JSON schemas defined in this standard and replaces the reference to the JSON schema 2020-12.

```
{
  "$schema": "https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/meta.json",
  // ...
}
```

The format validation is enforced by setting the corresponding vocabulary as required.

If a library used to parse, modify or create OpenEoX content is unable to deal with this meta schema, it could reach the objective by interpreting the schema as JSON schema 2020-12 dialect and enforcing the format validation via its implementation.

4.1.6 Date and Time

This standard uses the date-time format as defined in JSON Schema Draft 2020-12 Section 7.3.1. In accordance with [RFC3339] and [ISO8601-1], the following rules apply:

- The letter `T` separating the date and time SHALL be upper case.
- The separator between date and time MUST be the letter `T`.
- The letter `Z` indicating the timezone UTC SHALL be upper case.

- Fractions of seconds are allowed as specified in the standards mentioned above with the full stop (.) as separator.
- Empty timezones MUST NOT be used.
- The ABNF of [\[RFC3339\]](#), section 5.6 applies.

In contrast to the aforementioned standards, leap seconds MUST NOT be used.

While a full support of [\[RFC3339\]](#) would be preferred, significant challenges have been mentioned by implementers as most libraries are lacking the support for leap seconds. To ensure interoperability, the decision was made to prohibit leap seconds.

4.2 Changes From the Previous Version

The list of changes from the previous version and any revision history can be found in [Appendix 2](#).

5. Taxonomy

The following subsections describe the taxonomy defining and explaining all terms the standard is built upon.

5.1 End-of-Life

The End-of-Life (EoL) indicates the last day when the particular product (or the product version/release) is officially supported in any way by the vendor. After this date there shouldn't be more development, updates or any type of support expected from the vendor. The existing customers are also impacted by this product life cycle stage and should consider migration to the still supported product version or release.

5.2 End-of-Sales

The End-of-Sales (EoS) indicates the last day when a particular product (or the product version/release) may be ordered by customers from vendor sales channels. After this date, the product is no longer for sale. However, there might be other sources where the product is still available. Once the product reaches the EoS life cycle stage, it may still be supported by the vendor, based on the official or dedicated vendor life cycle model for this product, for existing customers. The implications for existing customers regarding license renewals, updates, upgrades to newer versions or ongoing technical support can vary depending on the vendor's specific policies.

5.3 End-of-Security-Support

The End-of-Security-Support (EoSSEC) indicates the last day when the vendor has committed to providing security remediations for the particular product (or the product version/release).

5.4 General Availability

The General Availability (GA) indicates the first day when the particular product (or the product version/release) is officially launched and made accessible to the general public or through its intended distribution channels.

5.5 Product

Product is any deliverable (e.g. software, hardware, specification, or service) which can be referred to with a name. This applies regardless of the origin, the license model, or the mode of distribution of the deliverable.

5.6 Product Life Cycle

Every product type (software, hardware, managed service and other deliverables) can be associated with a life cycle model. It can contain definitions of various support models (different levels of maintenance) in association to the product versioning convention. The life cycle support model can be dynamic and may change over time, from the product's initial release (General Availability) to its discontinuation (End-of-Life). During the product life cycle, support models may switch from one state to another and may even run in parallel to meet individual requirements. Those requirements may depend on the product type, the vendor offerings, as well as geographical related regulations.

5.7 Vendor

Vendor refers to the community, individual, or organization that created or maintains a product (software, hardware, managed service and other deliverables). This includes, but is not limited to, open-source software and hardware providers.

6. Schema Elements

The OpenEoX Core schema describes how to represent OpenEoX core information as a JSON document.

The OpenEoX Core schema Version 1.0 builds on the JSON Schema draft 2020-12 rules extended by the format validation enforcement (see 4.1.5).

```
"$schema": "https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/meta.json",
```

The schema identifier is:

```
"$id": "https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/core.json",
```

The further documentation of the schema is organized via Definitions and Properties.

- Definitions provide types that extend the JSON schema model
- Properties use these types to support assembling OpenEoX core information

Types and properties together provide the vocabulary for the domain specific language supporting OpenEoX information.

6.1 Definitions

The definitions (`$defs`) introduce the following domain specific types into the OpenEoX language: EoX Timestamp (`eox_timestamp_t`).

```
"$defs": {
  "eox_timestamp_t": {
    // ...
  }
},
```

6.1.1 EoX Timestamp Type

The EoX Timestamp (`eox_timestamp_t`) type of value type `string` contains the timestamp at which the product reaches the specified stage or the indicator that this is still ‘to be announced’.

```
"oneOf": [
  {
    "format": "date-time"
  },
  {
    "const": "tba"
  }
]
```

Therefore, it MUST either be a `string` with format `date-time` or use the constant value `tba`.

OpenEoX Consumer MUST treat `tba` as a date greater than the maximum date. In any semantic interpretation, the value `tba` MUST be treated as ‘not yet announced’.

6.2 Properties

The following subsections document the six properties of an OpenEoX Core JSON document. The four mandatory properties are `$schema`, `end_of_life`, `end_of_security_support`, and `last_updated`. The properties `end_of_sales` and `general_availability` are optional.

6.2.1 Schema Property

The OpenEoX Core Schema (`$schema`) of value type `string` and `const` specifies the schema the JSON object must be valid against. The single valid value for this `const` is:

```
https://docs.oasis-open.org/openeox/eox-core/v1.0/schema/core.json
```

This value allows for tools to identify that a JSON document is meant to be valid against this schema. Tools can use that to support users by automatically checking whether the OpenEoX Core Object adheres to the JSON schema identified by this URL.

6.2.2 End-of-Life Property

End-of-Life (`end_of_life`) of value type `eoX_timestamp_t` indicates the last day when the particular product is officially supported in any way by the vendor.

6.2.3 End-of-Sales Property

End-of-Sales (`end_of_sales`) of value type `eoX_timestamp_t` indicates the last day when a particular product may be ordered by customers from vendor sales channels.

6.2.4 End-of-Security-Support Property

End-of-Security-Support (`end_of_security_support`) of value type `eoX_timestamp_t` indicates the last day when the vendor has committed to providing security remediations for the particular product.

6.2.5 General Availability Property

General Availability (`general_availability`) of value type `eoX_timestamp_t` indicates the first day when the particular product is officially launched and made accessible to the general public or through its intended distribution channels.

The following special cases are defined:

- If the value of `general_availability` is equal to the value of `end_of_security_support`, this implies that the product is not supported. However, the product may be before its EoL.
- If a product is sunset (in terms of EoL) before its GA, the `general_availability` entry SHALL be removed and just the EoL entry kept in the OpenEoX record.
- If the value of `general_availability` is equal to the value of `end_of_life`, this implies that the product was published but is not maintained at all.

An example could be the publication of source code as open source without the intend to maintain it. The clear communication allows others to take over the development as well as to assess the risks associated with using the product.

6.2.6 Last Updated Property

Timestamp of last change (`last_updated`) of value type `string` with format `date-time` contains the RFC 3339 timestamp when the record was last updated.

7. Tests

The following subsections list a number of tests which all will have a short description and an excerpt of an example which fails the test.

7.1 Mandatory Tests

Mandatory tests MUST NOT fail at a valid EoX Core Information. A program MUST handle a test failure as an error.

7.1.1 Inconsistent EoL Date

It MUST be tested that the `end_of_life` is later than or equal to any other date value in the EoX Core Information. The property `last_updated` is ignored in this test. As the timestamps might use different timezones, the sorting MUST take timezones into account. Except for `end_of_life`, any property of type `eox_timestamp_t` with value `tba` MUST be ignored in the comparison.

The relevant path for this test is:

```
/end_of_life
```

Example 1 (which fails the test):

```
{
  // ...
  "end_of_life": "2025-11-19T16:00:00Z",
  "end_of_security_support": "2025-12-31T12:00:00Z",
  // ...
}
```

The `end_of_security_support` is later than the `end_of_life`.

7.1.2 Date and Time

For each item of type `string` and format `date-time` it MUST be tested that it conforms to the rules given in section 4.1.6. Any property with the value `tba` is ignored in this comparison.

The relevant path for this test is:

```
/end_of_life
/end_of_sales
/end_of_security_support
/general_availability
/last_updated
```

Example 1 (which fails the test):

```
"end_of_life": "2025-12-31 12:00:00Z",
```

The `end_of_life` uses a white space as separator instead the letter `T`.

7.1.3 Inconsistent GA Date

It MUST be tested that the `general_availability` is earlier than or equal to any other date value in the EoX Core Information. The property `last_updated` is ignored in this test. As the timestamps might use different timezones, the sorting MUST take timezones into account. The test MUST be skipped if the value of `general_availability` is `tba`. Except for `general_availability`, any property of type `eox_timestamp_t` with value `tba` MUST be ignored in the comparison.

The relevant path for this test is:

```
/general_availability
```

Example 1 (which fails the test):


```
{
  // ...
  "end_of_security_support": "2025-11-19T16:00:00Z",
  "general_availability": "2026-01-21T17:00:00Z",
  // ...
}
```

The `general_availability` is later than the `end_of_security_support`.

7.2 Recommended Tests

Recommended tests SHOULD NOT fail at a valid EoX Core Information without a good reason. Failing such a test does not make the EoX Core Information invalid. These tests may include information about features which are still supported but expected to be deprecated in a future version of OpenEoX. A program MUST handle a test failure as a warning.

7.2.1 Use of `tba` where Date is set for EoL

For each property of type `eoxtimestamp_t`, it MUST be tested that it does not contain the value `tba` if the `end_of_life` value is set to a date.

The relevant path for this test is:

```
/end_of_sales
/end_of_security_support
/general_availability
```

Example 1 (which fails the test):

```
{
  // ...
  "end_of_life": "2025-09-01T12:00:00Z",
  "end_of_security_support": "tba",
  // ...
}
```

The `end_of_security_support` is set to `tba` but `end_of_life` already has a date.

A tool MAY set the `end_of_security_support` to the same value as `end_of_life` as a quick fix.

7.2.2 Use of `tba` for GA where other Date is set

It MUST be tested that `general_availability` does not contain the value `tba` if any other property of type `eoxtimestamp_t` is set to a date.

The relevant path for this test is:

```
/general_availability
```

Example 1 (which fails the test):

```
{
  // ...
  "end_of_security_support": "2025-06-17T14:00:00Z",
  "general_availability": "tba",
  // ...
}
```

The `general_availability` is set to `tba` but `end_of_security_support` already has a date.

A tool MAY remove the `general_availability` as a quick fix.

7.3 Informative Tests

Informative tests provide insights in common mistakes and bad practices. They MAY fail at a valid EoX Core Information. It is up to the issuing party to decide whether this was an intended behavior and can be ignore or should be treated. These tests MAY include information about recommended usage. A program MUST handle a test failure as a information.

7.3.1 EoSSec before EoS

It MUST be tested that if `end_of_sales` is present, its value is before or equal to the value of `end_of_security_support`. As the timestamps might use different timezones, the sorting MUST take timezones into account. The test MUST be skipped, if `end_of_sales` is set to `tba`.

The relevant path for this test is:

```
/end_of_sales
```

Example 1 (which fails the test):

```
{  
  // ...  
  "end_of_sales": "2025-12-31T12:00:00Z",  
  "end_of_security_support": "2025-11-19T16:00:00Z",  
  // ...  
}
```

The `end_of_security_support` is earlier than the `end_of_sales`.

8. Safety, Security and Data Protection

All safety, security, and data protection requirements relevant to the context in which OpenEoX information items are used MUST be translated into, and consistently enforced through, OpenEoX implementations and processes.

OpenEoX information items are based on JSON, thus the security considerations of [\[RFC8259\]](#) apply and are repeated here as service for the reader:

Generally, there are security issues with scripting languages. JSON is a subset of JavaScript but excludes assignment and invocation.

Since JSON's syntax is borrowed from JavaScript, it is possible to use that language's `eval()` function to parse most JSON texts (but not all; certain characters such as U+2028 LINE SEPARATOR and U+2029 PARAGRAPH SEPARATOR are legal in JSON but not JavaScript). This generally constitutes an unacceptable security risk, since the text could contain executable code along with data declarations. The same consideration applies to the use of `eval()`-like functions in any other programming language in which JSON texts conform to that language's syntax.

9. Conformance

The single subsection of this section lists the conformance targets and clauses. The clauses directly corresponding to the targets are listed in separate sub-subsections of the target list subsection.

The order in which targets, and their corresponding clauses appear is somewhat arbitrary as there is no natural order on such diverse roles participating in the EoX exchanging ecosystem.

9.1 Conformance Targets

This document defines requirements for the file format and for certain software components that interact with it. The entities (“conformance targets”) for which this document defines requirements are:

- **EoX Core Information:** Information about end-of-everything in the format defined by this document.
- **Producer:** A program which emits output in the OpenEoX Core format.
- **Consumer:** A program that reads and interprets EoX Core Information.
- **Viewer:** An Consumer that reads EoX Core Information, displays a list of the results it contains, and allows an end user to view each result in the context of the artifact in which it occurs.
- **Viewer Recommending Actions:** A Viewer that provides actionable interpretations by mapping life cycle events to recommended actions.
- **Validator Basic:** A program that reads a JSON object and checks it against the JSON schema and performs mandatory tests.
- **Validator Extended:** A Validator Basic that additionally performs recommended tests.
- **Validator Full:** A Validator Extended that additionally performs guidance tests.
- **Library:** A library that implements OpenEoX Core data capabilities.
- **Library with Basic Validation:** A Library that also satisfies the conformance target “Validator Basic”.
- **Library with Extended Validation:** A Library that also satisfies the conformance target “Validator Extended”.
- **Library with Full Validation:** A Library that also satisfies the conformance target “Validator Full”.
- **Differ:** A program that compares EoX Core Information and calculates the changes.
- **Comparator:** A program that compares EoX Core Information and returns the newer one.
- **Sorter:** A program that sorts EoX Core Information from newest to oldest or vice versa.
- **Merger:** A program that combines EoX Core Information.

When advertising or mentioning the conformance targets, they **SHOULD** be prefixed with “OpenEoX Core” to avoid ambiguity as other parts of the OpenEoX specification might define similar conformance targets. For example, a “Viewer” from this specification **SHALL** be called “OpenEoX Viewer”. This does not apply for the EoX Core Information as this term is common over all parts of the specification.

9.1.1 Conformance Clause 1: EoX Core Information

A text file or data stream satisfies the “EoX Core Information” conformance profile if it:

- conforms to the syntax and semantics defined in section 4.1.5.
- conforms to the syntax and semantics defined in section 4.1.6.
- conforms to the syntax and semantics defined in section 6.
- does not fail any mandatory test defined in section 7.1.

9.1.2 Conformance Clause 2: Producer

A program satisfies the “Producer” conformance profile if the program:

- produces output in the OpenEoX Core format, according to the conformance profile “EoX Core Information”.
- satisfies those normative requirements in section 6 and 8 that are designated as applying to EoX Producers.

9.1.3 Conformance Clause 3: Consumer

A processor satisfies the “Consumer” conformance profile if the processor:

- reads EoX Core Information and interprets them according to the semantics defined in section 6.
- satisfies those normative requirements in section 6 and 8 that are designated as applying to EoX Consumers.

9.1.4 Conformance Clause 4: Viewer

A viewer satisfies the “Viewer” conformance profile if the viewer fulfills the following two groups of requirements:

The viewer:

- satisfies the “Consumer” conformance profile.
- satisfies the normative requirements given below.

For each value of type `/defs/eox_timestamp_t` that does not conform to [RFC3339] the viewer:

- SHOULD output the raw string value if the value is not `tba`.
- SHOULD display a value that is understandable to the user as a replacement for the technical value `tba`.

A tool MAY provide an option to suppress dates with value `tba`.

9.1.5 Conformance Clause 5: Viewer Recommending Actions

A viewer satisfies the “Viewer Recommending Actions” conformance profile if the viewer fulfills the following groups of requirements:

The viewer:

- satisfies the “Viewer” conformance profile.
- provides an interpretation view for each life cycle event that additionally answers:
 1. at what date the event occurs.
 2. what lifecycle state changes at that date.
 3. which actions are needed.

For this specification, the first two answers are directly derived from property semantics and their values. The third answer (needed actions) is not explicitly represented by a dedicated EoX Core property and therefore requires a defined action mapping by specification, profile, implementation, or deployment policy.

- still provides the date and change interpretation, if no action mapping is available.

The Viewer Recommending Actions SHOULD:

- indicate that the action is not specified by this specification, if no action mapping is available.
- use the following action mapping as default:
 - EoS: Product can no longer be ordered from vendor sales channels. *Recommended action:* evaluate and initiate replacement procurement plans for successor offerings.
 - EoSSec: Vendor no longer commits to security remediations. *Recommended action:* complete upgrade or migration to a supported product version and apply risk treatment until migration is complete.
 - EoL: All vendor support ends. *Recommended action:* complete replacement or decommissioning of affected deployments.

An Viewer Recommending Actions MAY use different action mappings or expand on the recommended actions.

9.1.6 Conformance Clause 6: Validator Basic

A program satisfies the “Validator Basic” conformance profile if the program:

- reads JSON objects and performs a check against the JSON schema.
- performs all mandatory tests as given in section 7.1.
- does not change the EoX Core Information.

A Validator Basic MAY provide one or more additional functions:

- Only run one or more selected mandatory tests.
- Apply quick fixes as specified in the standard.
- Apply additional quick fixes as implemented by the vendor.

9.1.7 Conformance Clause 7: Validator Extended

A Validator Basic satisfies the “Validator Extended” conformance profile if the Validator Basic:

- satisfies the “Validator Basic” conformance profile.
- additionally performs all recommended tests as given in section 7.2.
- provides a function that allows to specify tests for which test results of warning are reported as error.

A Validator Extended MAY provide an additional function to only run one or more selected recommended tests.

9.1.8 Conformance Clause 8: Validator Full

A Validator Extended satisfies the “Validator Full” conformance profile if the Validator Extended:

- satisfies the “Validator Extended” conformance profile.
- additionally performs all guidance tests as given in section 7.3.
- provides a function that allows to specify tests for which test results of information are reported as warning.
- provides a function that allows to specify tests for which test results of information are reported as error.

A Validator Full MAY provide an additional function to only run one or more selected guidance tests.

9.1.9 Conformance Clause 9: Library

A library satisfies the “Library” conformance profile if the library:

- implements all elements as data structures conforming to the syntax and semantics defined in sections 4.1.5, 4.1.6 and 6.
- checks all elements according to the patterns provided in the JSON schema.
- has a function that reads EoX Core Information into the data structure from a
 - file system.
 - URL.
 - data stream.
- provides function for sorting the keys and sorts the keys automatically on output.
- has a function that outputs the data structure as EoX Core Information
 - on the file system.
 - as string.
 - into a data stream.

The library MAY implement an option to retrieve the keys unsorted.

9.1.10 Conformance Clause 10: Library with Basic Validation

A Library satisfies the “Library with Basic Validation” conformance profile if the Library:

- satisfies the “Library” conformance profile.
- satisfies the “Validator Basic” conformance profile.
- validates the EoX Core Information before output according to the “Validator Basic” and presents the validation result accordingly.
- provide a function to validate the data structure in its current state according to the “Validator Basic” and presents the validation result accordingly.

A Library does not satisfies the “Library with Basic Validation” conformance profile if the EoX Core Library uses an external library or program for the “Validator Basic” part and does not enforce its presence.

9.1.11 Conformance Clause 11: Library with Extended Validation

A Library satisfies the “Library with Extended Validation” conformance profile if the Library:

- satisfies the “Library” conformance profile.
- satisfies the “Validator Extended” conformance profile.
- validates the EoX Core Information before output according to the “Validator Extended” and presents the validation result accordingly.
- provide a function to validate the data structure in its current state according to the “Validator Extended” and presents the validation result accordingly.

A Library does not satisfies the “Library with Extended Validation” conformance profile if the EoX Core Library uses an external library or program for the “Validator Extended” part and does not enforce its presence.

9.1.12 Conformance Clause 12: Library with Full Validation

A Library satisfies the “Library with Extended Validation” conformance profile if the Library:

- satisfies the “Library” conformance profile.
- satisfies the “Validator Full” conformance profile.
- validates the EoX Core Information before output according to the “Validator Full” and presents the validation result accordingly.
- provide a function to validate the data structure in its current state according to the “Validator Full” and presents the validation result accordingly.

A Library does not satisfies the “Library with Full Validation” conformance profile if the EoX Core Library uses an external library or program for the “Validator Full” part and does not enforce its presence.

9.1.13 Conformance Clause 13: Differ

A program satisfies the “Differ” conformance profile if the program:

- satisfies the “Consumer” conformance profile.
- calculates the difference between two EoX Core Information JSON Objects and presents it as JSON Diff.

A Differ MAY choose to make that information also available in other data formats.

9.1.14 Conformance Clause 14: Comparator

A program satisfies the “Comparator” conformance profile if the program:

- satisfies the “Consumer” conformance profile.
- provides a function to compares two EoX Core Information Objects *a* and *b* based on their `last_updated` value and returns:
 - ▶ -1 if `a["last_updated"] < b["last_updated"]`
 - ▶ 0 if `a["last_updated"] = b["last_updated"]`
 - ▶ 1 if `a["last_updated"] > b["last_updated"]`
- provides a function to compares two EoX Core Information Objects *a* and *b* based on their `last_updated` value and returns:
 - ▶ *b* if `a["last_updated"] < b["last_updated"]`
 - ▶ *a* if `a["last_updated"] = b["last_updated"]`
 - ▶ *a* if `a["last_updated"] > b["last_updated"]`
- takes timezones into account.

A Comparator MAY choose to make that information also available in other data formats.

9.1.15 Conformance Clause 15: Sorter

A program satisfies the “Sorter” conformance profile if the program:

- satisfies the “Consumer” conformance profile.
- provides a function to sort an arbitrary number of EoX Core Information Objects `last_updated` value with that value descending (default).
- provides a function to sort an arbitrary number of EoX Core Information Objects `last_updated` value with that value ascending.
- takes timezones into account.

A Sorter MAY choose to make that information also available in other data formats.

9.1.16 Conformance Clause 16: Merger

A program satisfies the “Merger” conformance profile if the program:

- satisfies the “Consumer” conformance profile.
- provides a function to merge EoX Core Information into a single EoX Core Information Object by overwriting all old values of keys present in newer EoX Core Information Objects with the values from newer objects.

This includes the value of `last_updated`.

- provides a function to merge EoX Core Information into a single EoX Core Information Object by overwriting all old values of keys present in newer EoX Core Information Objects with the values from newer objects but treating `tba` as not present during the overwrite process.

This includes the value of `last_updated`. Ignoring `tba` while overwriting ensures that a date value is never overwritten by `tba`.

- provides a function to merge EoX Core Information into a single EoX Core Information Object by taking the newest as base and iterative appending keys that are not present in that object while iterating over the EoX Core Information from newest to oldest.

As `last_updated` is a required field, it is not changed during the process. Values of keys that are inserted are never changed in that process.

- takes timezones into account.

A program MAY implement other algorithms than defined here, if and only if, the results are guaranteed to be the same.

A Merger MAY choose to make that information also available in other data formats.

Annex A. License, Document Status and Notices

(This annex forms an integral part of this Specification.)

A.1 Document Status

This document was last revised or approved by the OASIS OpenEoX TC on the above date. The level of approval is also listed above. Check the “Latest version” location noted above for possible later revisions of this document. Any other numbered Versions and other technical work produced by the Technical Committee (TC) are listed at <https://groups.oasis-open.org/communities/tc-community-home2?CommunityKey=26350f39-9c7b-4bf2-a422-018dc7d3f5aa>.

TC members should send comments on this document to the TC’s email list. Others should send comments to the TC’s public comment list, after subscribing to it by following the instructions at the “Send A Comment” button on the TC’s web page at <https://www.oasis-open.org/committees/openeox/>.

NOTE: any machine-readable content (Computer Language Definitions) declared Normative for this Work Product is provided in separate plain text files. In the event of a discrepancy between any such plain text file and display content in the Work Product’s prose narrative document(s), the content in the separate plain text file prevails.

A.2 License and Notices

Copyright © OASIS Open 2026. All Rights Reserved.

All capitalized terms in the following text have the meanings assigned to them in the OASIS Intellectual Property Rights Policy (the “OASIS IPR Policy”). The full Policy, which governs the licensure of this document, may be found at the OASIS website: [<https://www.oasis-open.org/policies-guidelines/ipr/>]

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to OASIS, except as needed for the purpose of developing any document or deliverable produced by an OASIS Technical Committee (in which case the rules applicable to copyrights, as set forth in the OASIS IPR Policy, must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by OASIS or its successors or assigns, as provided in the OASIS IPR Policy.

This document is provided under the “Non-Assertion” IPR mode that was chosen when the project was established, as defined in the IPR Policy. For information on whether any patents have been disclosed that may be essential to implementing this document, and any offers of patent licensing terms, please refer to the Intellectual Property Rights section of the project’s web page (<https://www.oasis-open.org/committees/openeox/ipr.php>).

This document and the information contained herein is provided on an “AS IS” basis and OASIS DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. OASIS AND ITS MEMBERS WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THIS DOCUMENT OR ANY PART THEREOF.

As stated in the OASIS IPR Policy, the following three paragraphs in brackets apply to OASIS Standards Final Deliverable documents (Committee Specifications, OASIS Standards, or Approved Errata).

OASIS requests that any OASIS Party or any other party that believes it has patent claims that would necessarily be infringed by implementations of this OASIS Standards Final Deliverable, to notify OASIS TC Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the OASIS Technical Committee that produced this deliverable.

OASIS invites any party to contact the OASIS TC Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this OASIS Standards Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the OASIS Technical Committee that produced this OASIS Standards Final Deliverable. OASIS may include such claims on its website, but disclaims any obligation to do so.

OASIS takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this OASIS Standards Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on OASIS' procedures with respect to rights in any document or deliverable produced by an OASIS Technical Committee can be found on the OASIS website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this OASIS Standards Final Deliverable, can be obtained from the OASIS TC Administrator. OASIS makes no representation that any information or list of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.

The name "OASIS" is a trademark of OASIS, the owner and developer of this document, and should be used only to refer to the organization and its official outputs. OASIS welcomes reference to, and implementation and use of, its documents, while reserving the right to enforce its marks against misleading uses. Please see <https://www.oasis-open.org/policies-guidelines/trademark/> for guidance.

Annex B. References

(This annex forms an integral part of this Specification.)

This section contains the normative and informative references that are used in this document.

Normative references are specific (identified by date of publication and/or edition number or version number) and Informative references are either specific or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the reference document (including any amendments) applies. While any hyperlinks included in this section were valid at the time of publication, OASIS cannot guarantee their long term validity.

B.1 Normative References

The following documents are referenced in such a way that some or all of their content constitutes requirements of this document.

[**ECMA-262**] *ECMAScript® 2024 Language Specification*, ECMA-262, 15th edition, June 2024, <https://262.ecma-international.org/15.0/>.

[**ISO8601-1**] *Date and time — Representations for information interchangePart 1: Basic rules*, International Standard, ISO 8601-1:2019(E), February 25, 2019, <https://www.iso.org/standard/70907.html>.

[**JSON-Schema-Core**] *JSON Schema: A Media Type for Describing JSON Documents*, draft-bhutton-json-schema-00, December 2020, <https://datatracker.ietf.org/doc/html/draft-bhutton-json-schema-00>.

[**JSON-Schema-Validation**] *JSON Schema Validation: A Vocabulary for Structural Validation of JSON*, draft-bhutton-json-schema-validation-00, December 2020, <https://datatracker.ietf.org/doc/html/draft-bhutton-json-schema-validation-00>.

[**JSON-Hyper-Schema**] *JSON Hyper-Schema: A Vocabulary for Hypermedia Annotation of JSON*, draft-handrews-json-schema-hyperschema-02, September 2019, <https://json-schema.org/draft/2019-09/json-schema-hypermedia.html>.

[**Relative-JSON-Pointers**] *Relative JSON Pointers*, draft-bhutton-relative-json-pointer-00, December 2020, <https://datatracker.ietf.org/doc/html/draft-bhutton-relative-json-pointer-00>.

[**RFC2119**] Bradner, S., “Key words for use in RFCs to Indicate Requirement Levels”, BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[**RFC3339**] Klyne, G. and C. Newman, “Date and Time on the Internet: Timestamps”, RFC 3339, DOI 10.17487/RFC3339, July 2002, <https://www.rfc-editor.org/info/rfc3339>.

[**RFC4180**] Shafranovich, Y., “Common Format and MIME Type for Comma-Separated Values (CSV) Files”, RFC 4180, DOI 10.17487/RFC4180, October 2005, <https://www.rfc-editor.org/info/rfc4180>.

[**RFC7230**] Roy T. Fielding and Julian Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing”, RFC 7230, DOI 10.17487/RFC7230, June 2014, <https://www.rfc-editor.org/info/rfc7230>.

[**RFC7464**] Williams, N., “JavaScript Object Notation (JSON) Text Sequences”, RFC 7464, DOI 10.17487/RFC7464, February 2015, <https://www.rfc-editor.org/info/rfc7464>.

[**RFC8174**] Leiba, B., “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words”, BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[**RFC8259**] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format”, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/info/rfc8259>.

[**RFC9562**] Davis, K., Peabody, B., and P. Leach, “Universally Unique IDentifiers (UUIDs)”, RFC 9562, DOI 10.17487/RFC9562, May 2024, <https://www.rfc-editor.org/info/rfc9562>.

[**SPDX301**] *The System Package Data Exchange® (SPDX®) Specification Version 3.0.1*, Linux Foundation and its Contributors, 2024, <https://spdx.github.io/spdx-spec/>.

B.2 Informative References

The following referenced documents are not required for the application of this document but may assist the reader with regard to a particular subject area.

[**CSAF-v2.1**] *Common Security Advisory Framework Version 2.1*. Edited by Stefan Hagen, and Thomas Schmidt. 28 May 2025. OASIS Committee Specification Draft 01. <https://docs.oasis-open.org/csaf/csaf/v2.1/csd01/csaf-v2.1-csd01.html>. Latest stage: <https://docs.oasis-open.org/csaf/csaf/v2.1/csaf-v2.1.html>.

Appendix 1. Acknowledgments

(This appendix does not form an integral part of this Specification and is informational.)

The following individuals were members of the OASIS OpenEoX Technical Committee during the creation of this specification and their contributions are gratefully acknowledged:

Leadership

The following individuals have had significant leadership positions during the development of this document, not just this version of the document, and they are gratefully acknowledged:

- Chairs:
 - Co-Chair, Justin Murphy (justin.murphy@cisa.dhs.gov), [DHS Cybersecurity and Infrastructure Security Agency \(CISA\)](#)
 - Co-Chair, Omar Santos (osantos@cisco.com), [Cisco Systems](#)
- Secretaries:
 - Secretary, Stefan Hagen (stefan@hagen.link), [Individual](#)
- Editors
 - Editor, Jautau White (jaywhite@microsoft.com), [Microsoft Corporation](#)
 - Editor, Stefan Hagen (stefan@hagen.link), [Individual](#)
 - Editor, Thomas Schmidt (thomas.schmidt@bsi.bund.de), [Federal Office for Information Security \(BSI\) Germany](#)

Special Thanks

The following individuals have made substantial contributions to this document, not just this version of the document, and their contributions are gratefully acknowledged:

- < Will be filled in at CS01 >

Participants

The following individuals were members of this committee during the creation of this document, not just this version of the document, and their contributions are gratefully acknowledged:

- < Will be filled in at CS01 >

Appendix 2. Changes From Previous Version

(This appendix does not form an integral part of this Specification and is informational.)

N/A.

Revision History

Revision tracking is publicly available in the [version control system](#).